PENNYPILOT — PRODUCT STRATEGY & ENGINEERING BLUEPRINT

Prepared for the NIVUNA Labs Application Development Assignment. Blueprint only, no application code.

0. Read this first: verification status and hard truths

What I did not verify. I did not open the three design repositories (claudedesignskills, taste-skill, impeccable) in this pass. Section B.3 therefore describes *how to use design-guidance repos in general* and what to check in them. Nothing here claims knowledge of their contents. Free-tier limits and model names change often; every such figure below is a **to-verify** item before you commit.

Hard truths about the assignment (inference, not fact):

1. The evaluators are grading *engineering judgement and your ability to explain it*, not feature count. A smaller app you can defend line by line beats a large one you cannot. Your stated rubric (problem understanding, architecture, AI workflow, challenges) is mostly about explanation.
2. "Premium fintech SaaS" in 3–4 days is achievable only with a tight design system and ruthless scope. Beautiful and broad are in tension. Choose beautiful and narrow.
3. Your own feature list (A–I) is about 3x what can be built *and tested* in the window. Cut to roughly one third.
4. The single biggest risk is not technical. It is a demo that breaks live. Plan for reliability first, delight second.
5. Do not over-index on "SaaS". Multi-tenant auth, billing and scale are irrelevant to the grade. Correct user isolation is relevant.

A. Product strategy

Problem. Most people do not abandon expense trackers because they lack charts. They abandon them because *logging is tedious* and *insights are generic*. Entering "milk ₹60" through a form with five fields, daily, fails within two weeks. Dashboards then show totals the user already feels, without saying what to do.

Target user (assumption, state it as such in the demo). An urban Indian salaried worker or student, 18–35, UPI-heavy, with many small payments, no budgeting discipline, and no appetite for bank-linking or paid apps. The founder's examples (milk, breakfast) point at exactly this user.

Value proposition. *Say what you spent in one sentence; get verified numbers and plain-language guidance back.* Fast capture, trustworthy math, useful interpretation.

What differs from existing trackers.

Typical tracker weakness	PennyPilot response
Form-heavy manual entry	Natural-language capture with a confirm step
Generic category pie charts	Charts tied to a question: "where did this month diverge from my normal?"
AI that guesses numbers	Deterministic engine computes; LLM only narrates a verified fact bundle
Alerts that nag	Few, explained, dismissible signals
Cloud-only black box	Works fully without AI; deterministic fallbacks everywhere

Three strongest differentiators (my ranking):

1. **Natural-language entry with review-before-save** (multi-item, handles Indian amounts, relative dates). Highest user value, very demo-friendly, genuinely benefits from a language model plus a good parser.
2. **Grounded Financial Copilot** that answers questions over *the user's own data* via a fixed set of backend query functions (not free-form SQL, not LLM arithmetic). Directly satisfies your "no fabricated calculations" rule and is the best discussion topic.
3. **Insight engine with explanations** (anomalies + budget pace + recurring detection), all rule/statistics-based, each insight carrying its evidence. Optional LLM rephrasing. This shows engineering maturity: AI where needed, algorithms where sufficient.

B. Competitive and design analysis

B.1 Common weaknesses of conventional trackers

- Entry friction (many taps per transaction).
- Category rigidity; "Other" becomes 40% of data.
- Backward-looking totals without baselines ("₹9,000 on food" means nothing without "your usual is ₹6,500").
- Budgets that only turn red after the damage is done; no *pace* awareness (spent 70% in 40% of the month).
- Insights that are templated platitudes.
- Visual noise: gradients, many donut charts, ten KPIs of equal weight.

B.2 Opportunities and honest feature triage

Useful: pace-based budgets, baseline comparisons, recurring detection, confirm-before-save capture. Visual complexity to avoid: 3D charts, animated gradient blobs, glassmorphism everywhere, more than one hero chart per screen, dashboards with 10+ widgets.

B.3 Design-reference repos: how to use them (not verified)

You listed three repos. I have not inspected them, and I will not infer contents from names. What to do:

- Open each; read the README and any rules or skill files. Extract only **concrete, checkable rules** (type scale, spacing system, contrast thresholds, anti-pattern lists).
- Convert the useful rules into a single project file your AI coding tool reads on every task (a design-rules markdown in the repo root, referenced through your tool's rules/instructions mechanism). Where advice conflicts, your design tokens win.
- Treat these as *constraints and a review checklist*, not as generators. In the presentation, say which rules you adopted and which you rejected, and why. That is stronger evidence of judgement than "I used a design skill".
- Check licences before copying any text or assets.

I can fetch and summarize them if you want; that would be a separate step with real inspection.

C. Functional requirements

C.1 Feature evaluation

Feature	User value	Feasibility	Time	Demo impact	Verdict
Manual CRUD transactions	Essential	High	Low	Low	**MVP**
Categories (defaults + custom)	Essential	High	Low	Low	**MVP**
Budgets per category (monthly)	High	High	Low-Med	Med	**MVP**
Dashboard (totals, trend, category)	High	High	Med	High	**MVP**
NL entry + confirm	Very high	Med	Med	Very high	**MVP differentiator**
Insight engine (anomaly + pace)	High	Med	Med	High	**Differentiator**
AI Copilot (grounded Q&A)	High	Med	Med-High	Very high	**Differentiator** (constrained)
AI monthly report	Med	High (reuses insights)	Low	Med	**Differentiator, cheap**
Recurring detection	Med-High	Med	Med	Med	**Optional, one simple version**
CSV import	Med	Med	Med	Med	**Optional**; export is cheaper
CSV export	Med	High	Low	Low	**Optional, cheap**
What-if simulator	Med	Med	Med	High	**Optional**; simple arithmetic, no LLM needed
Financial goals	Med	High	Low-Med	Med	Only if simulator is built
Receipt OCR, bank/UPI linking, multi-currency, sharing, push notifications, investments	Varies	Low	High	Varies	**Avoid**

C.2 Tiers

Must-have MVP: auth, transactions CRUD with filters, categories, monthly budgets with utilization, dashboard, NL entry with review, seed demo data, deterministic fallbacks.

High-impact differentiators (pick these three): insight engine (budget pace + anomalies), grounded Copilot, monthly AI report.

Optional (only if ahead of schedule): CSV export, recurring detection (simple), what-if simulator.

Avoid within the deadline: CSV import with auto-categorization, OCR, bank linking, multi-currency, notifications, offline sync, social features, any microservice.

C.3 Key user workflows

W1 Natural-language entry.

1. User types "₹120 milk and ₹450 groceries today, auto 80 yesterday".
2. Backend parses to a draft list: amount, description, category suggestion, date, type, confidence.
3. UI shows an editable review table; low-confidence rows are flagged.
4. User confirms; backend validates and writes in one DB transaction.
5. Dashboard updates; budget impact is shown inline ("Groceries now 74% used").

The user never loses typed text on failure; parser failure falls back to a prefilled manual form.

W2 Dashboard review. Open app → verdict strip (one sentence: "On pace to overspend Food by about ₹1,800") → hero chart (cumulative spend vs. budget pace) → category breakdown → insight cards → recent transactions.

W3 Budget setting. Choose month → set per-category amounts (suggestion shown: median of last 3 months, computed) → save → see utilization and projected month-end.

W4 Copilot question. User asks "Why was food higher this month?" → backend selects query tools → fact bundle → LLM writes answer → UI shows the answer plus an **Evidence** panel listing the exact figures and transactions used.

W5 Monthly report. Pick month → backend builds report facts → narrative generated (or template if AI is down) → view, print or export.

D. System architecture

D.1 Recommended stack

Layer	Recommendation	Why	Alternatives and why not
Frontend	**React + TypeScript + Vite**, Tailwind CSS with a custom token layer, TanStack Query, React Hook Form + Zod	Fast SPA for a data-heavy, interactive app; TypeScript catches errors AI tools introduce	Next.js: SSR and routing power you do not need; more deploy overhead. Plain HTML/JS: weak for this app
Charts	**Recharts** for most charts, custom SVG (or visx/D3) for the one signature chart	Free, React-native; the custom chart gives distinctiveness	Chart.js: fine, less React-idiomatic. ECharts: powerful but heavy and harder to style premium
Backend	**FastAPI (Python) + Pydantic**	Python suits AI work and your ML background; Pydantic gives validation and auto OpenAPI docs you can show live	Node/Express: fine, but loses Pydantic-style schema rigor and Python ML tooling. Django: heavier than needed
Database	**SQLite locally; PostgreSQL (free-tier managed) for the hosted demo** via SQLAlchemy + Alembic	SQLite makes local demo and tests trivial; same ORM lets you swap	Mongo: financial data is relational and aggregates are SQL's strength. Firebase: lock-in, weaker aggregation
Auth	**Email + password, argon2/bcrypt hashing, short-lived JWT in an httpOnly cookie** (or server sessions)	Real isolation with few moving parts	Third-party auth: free tiers exist but add dependency and limits. Skip OAuth

Layer	Recommendation	Why	Alternatives and why not
AI	**Provider-abstracted LLM client** with ordered fallbacks: (1) free-tier hosted API, (2) local model via Ollama, (3) deterministic rules/templates	Satisfies "works when AI is down"	Single provider: single point of failure in a live demo
Deployment	Frontend on a free static host; backend on a free-tier container host	Verify current limits first	See D.4

Why not microservices, queues, Redis, a vector DB? None is needed. A single FastAPI service with modules is the right size. If asked: "Boundaries are modules, not services; I would split only for a scaling or team-ownership reason."

D.2 Backend module layout (conceptual)

- **api/** routers (auth, transactions, categories, budgets, insights, copilot, reports, NL entry)
- **domain/** pure Python financial logic (totals, aggregates, pace, anomalies, recurring). No I/O, no AI. The heart of the system and the most tested part.
- **ai/** LLM client, prompt templates, output schemas, fallback chain
- **db/** models, repositories, migrations
- **core/** config, security, error types, logging

Key rule: **AI modules receive only output from `domain/`; they never compute totals.**

D.3 API design (REST, JSON, versioned `/api/v1`)

- `POST /auth/register`, `/auth/login`, `/auth/logout`, `GET /auth/me`
- `GET/POST /transactions`, `PATCH/DELETE /transactions/{id}`; filters: date range, category, type, text, amount range; cursor pagination
- `POST /transactions/parse` (NL → drafts, **does not save**) and `POST /transactions/bulk` (confirmed drafts, atomic)
- `GET/POST/PATCH/DELETE /categories`

- `GET/PUT /budgets?month=YYYY-MM`
- `GET /dashboard/summary?month=`, `/dashboard/trends`, `/dashboard/category-breakdown`
- `GET /insights?month=` (computed, cached), `POST /insights/{id}/dismiss`
- `POST /copilot/ask`
- `GET /reports/monthly?month=`; `GET /export/transactions.csv`
- Money as integer **paise** in API and DB, formatted only in the UI. Dates as ISO strings with a stored user timezone (default Asia/Kolkata).
- Uniform error shape: `{code, message, field_errors?}`; OpenAPI docs exposed for the demo.

D.4 Free hosting (to verify before relying)

I have not checked current terms. Common patterns: static hosts (Vercel, Netlify, Cloudflare Pages) for the frontend; Render, Fly.io, Railway or similar for the backend; Neon, Supabase or similar for Postgres. Typical free-tier hazards: **cold starts** (first request can take tens of seconds), sleeping databases, expiring or credit-limited plans, and rate limits. **Mitigation for a live demo:** run the demo from your laptop (local backend, local DB) as the primary path, and keep the deployed URL as a backup that you warm up 10 minutes before. Never make the demo depend on a cold free tier.

E. Database design

All monetary values: integer paise. All tables except `users` carry `user_id`, and every query filters on it.

users: id (UUID), email (unique, lowercased), password_hash, display_name, timezone, currency (fixed INR for MVP), created_at.

categories: id, user_id, name, kind (expense | income), icon, color_token, is_archived, sort_order. Unique (user_id, name, kind). *Copy default categories per user at signup* instead of sharing a global table: simpler isolation, and users can rename freely.

transactions: id, user_id, category_id (FK, restrict delete; archive instead), kind (expense | income), amount_paise (> 0), occurred_on (date), description, merchant (nullable), note, source (manual | nl | csv), nl_original_text (nullable, for audit), recurring_id (nullable),

created_at, updated_at, deleted_at (soft delete). Indexes: (user_id, occurred_on DESC), (user_id, category_id, occurred_on), (user_id, kind, occurred_on), optional lowercase index on description for search.

budgets: id, user_id, category_id, month (first-of-month date), limit_paise. Unique (user_id, category_id, month). Decide before building whether the overall monthly budget is a separate field or a null-category row.

goals (only if the simulator is built): id, user_id, name, target_paise, target_date, saved_paise (derived or tracked; decide), created_at.

recurring_rules (optional): id, user_id, category_id, description, amount_paise, cadence (monthly | weekly), next_due, detected_or_manual, is_active.

insights: id, user_id, month, type (budget_pace | anomaly | category_shift | recurring), severity, facts_json (the verified numbers), template_text (deterministic sentence), ai_text (nullable), dismissed_at, created_at. Storing `facts_json` makes every insight auditable.

ai_interactions (history and audit): id, user_id, feature (copilot | report | parse), input_summary, facts_json, output_text, model_used, fallback_used (bool), latency_ms, validation_passed (bool), created_at. Keep text minimal; never log secrets.

Data ownership and isolation. Every repository function takes `user_id` as a required argument and applies it; no function queries a user-owned table without it. IDs are UUIDs (not sequential) as defense in depth. On Postgres, row-level security is a strong optional extra, but app-level enforcement plus tests that attempt cross-user access is sufficient and explainable.

Design decisions to defend: integer paise avoids float error; soft delete protects against accidental loss and keeps aggregates auditable; per-user default categories avoid a shared mutable table.

F. AI engineering strategy

F.1 Governing principle

Compute, then explain. Pipeline: question → deterministic query functions → structured **fact bundle** → LLM prompt containing only the bundle → structured output → **validator** checks every number in the output against the bundle → accept, or fall back to template text.

F.2 Feature-by-feature

Feature	Input	Processing	Output	LLM needed?	Failure handling	Validation
NL entry	Free text, user timezone, today's date, user's categories	Stage 1: deterministic rule parser for amounts (₹, Rs, "k", lakh), dates (today, yesterday, weekday names), keyword→category map from user history. Stage 2: LLM only for residual ambiguity (multi-item splitting, category choice) with schema-constrained JSON	Draft transactions with confidence	**Partly.** Rules handle simple cases; LLM helps messy input	If the LLM fails, return rule-based drafts with low-confidence flags, or a prefilled manual form	Schema check; amount > 0 and under a sane cap; date not far in future; category must exist; **user must confirm**
Categorization suggestion	Description	Merchant/keyword dictionary + user's previous description→category mappings (frequency-based); LLM only for unknowns	Category + confidence	**No** for most; history-based is better and learns from the user	Default to "Uncategorized"	Category ∈ user's categories
Budget pace and alerts	Transactions, budgets, today	Linear projection: spent-to-date ÷ elapsed days × days in month; compare with limit; thresholds (e.g. 80%, projected overrun)	Status, projected overrun, days-to-limit	**No**	n/a	Unit tests with known fixtures
Anomaly detection	Per-category history	Robust statistics: median and MAD (or IQR) per category; flag transactions far above baseline; flag category-month totals far above rolling baseline. Require minimum history, otherwise say "not enough data"	Flagged items with baseline numbers	**No** (LLM may only phrase the explanation)	Cold start: suppress, do not guess	Synthetic-spike tests; cap on false positives per month
Recurring detection	Transactions by normalized description	Group by normalized merchant; similar amount (±5%) and regular interval (about 30 or 7 days) over at least 3 occurrences	Candidate recurring items, next expected date	**No**	Shown as "suggested"; user confirms	Fixtures for monthly, weekly and noisy cases

Feature	Input	Processing	Output	LLM needed?	Failure handling	Validation
Copilot	User question	Router maps the question to a **fixed whitelist of query tools** (spend by category/period, compare periods, top merchants, budget status, list transactions by filter). LLM picks tool and arguments via schema, or a keyword/intent router does it when AI is down. Backend executes; LLM writes the answer from results	Answer + evidence panel	**Yes**, for intent and phrasing	If no tool matches or AI is down: show a menu of supported questions with direct deterministic answers. Never answer without evidence	Tool args validated; numbers in answer must appear in evidence; out-of-scope refused politely; no free-form SQL
What-if simulator	Purchase amount, date, goal, savings rate	Pure arithmetic projection	Impact table and chart	**No**	n/a	Unit tests
Monthly report	Month's fact bundle	Template report always generated; LLM optional narrative polish	Report	Optional	Template is default and fallback	Same number-check validator

Where an LLM is unnecessary (say this in the presentation): anomalies, budget pace, recurring detection, history-based categorization, simulator. Using an LLM there would be slower, less reliable and harder to test. This is your strongest "I know when not to use AI" point.

F.3 Free AI options (all to verify)

- **Hosted free tiers:** Google Gemini API, Groq, OpenRouter free models, Cloudflare Workers AI, Hugging Face Inference. Limits (requests per minute/day, context size, model availability, data-use terms) change; **check each provider's current page**. Some free tiers may use prompts for model improvement; read the terms and tell users.
- **Local:** Ollama with a small instruction-tuned model; needs RAM or a GPU you may not have, and is slow on CPU. Good as an offline fallback and a talking point; do not make it the only path.
- **Recommended chain:** hosted free provider → second provider or local model → deterministic fallback, with a per-request timeout (about 8–10 s) and a visible "AI offline, showing computed summary" state.

F.4 Hallucination controls

1. The LLM never sees raw data it could re-aggregate; only the fact bundle.
2. Structured outputs; reject on schema failure; one retry, then fallback.
3. Numeric validator: extract numbers from the output and require each to match a bundle value within rounding.
4. Prompt forbids advice beyond the facts (no investment, tax or legal advice); suggestions phrased as "consider".
5. Every AI-generated item carries an evidence link and an "AI-generated" label.
6. Log fallbacks and validation failures in `ai_interactions` for your challenges slide.

G. UI/UX architecture

G.1 Sitemap

- **Auth:** Sign in, Create account
- **Home (Dashboard)**
- **Add** (global quick-add; keyboard shortcut on desktop, floating action on mobile)
- **Transactions:** list, filters, search, edit drawer
- **Budgets:** month selector, category rows with pace markers
- **Insights:** active signals, dismissed history
- **Ask (Copilot)**
- **Reports:** monthly report view, export
- **Settings:** profile, categories, data export, delete account (even a stub shows maturity)
- Optional: **Plan** (what-if, goals)

G.2 Navigation

Desktop: slim left rail (icons + labels) with the quick-add input pinned at the top of the main area. Mobile: bottom tab bar (Home, Transactions, Add, Budgets, Ask) with a bottom-sheet quick-add. Fewer than six top-level destinations.

G.3 Dashboard composition (visual hierarchy)

1. **Verdict strip** (largest type): one computed sentence and the month selector. Shape: "₹X spent, ₹Y left, on pace to finish ₹Z over/under."
2. **Hero chart:** cumulative spend through the month versus the budget pace line, with last month as a faint comparison line. This single chart carries the product's idea.
3. **Two-column row:** category breakdown as sorted horizontal bars (not a donut) with budget markers; upcoming/recurring list or top changes versus baseline.
4. **Insight cards (max three)**: headline, one-line evidence, one action.
5. **Recent transactions** (compact list).

One accent colour per region; everything else neutral. This discipline separates "commercial" from "student".

G.4 Recommended design direction: "Quiet Ledger"

A calm, editorial, numbers-first look, deliberately unlike neon-gradient dashboards.

- **Palette (light default, dark supported):** warm off-white surface (around #F7F5F0), ink text (around #14161A), deep green-teal as the single brand accent (around #0F766E), amber for caution, muted brick red for overspend, soft sage for positive. Never rely on colour alone for status (pair with icon or label). Define as CSS custom properties; check every text/background pair for WCAG AA contrast (4.5:1 for body text).
- **Typography:** a characterful display serif or high-contrast sans for the verdict figure and headings (free Google Fonts options such as Fraunces, Newsreader or Instrument Serif; confirm licences on the font page) paired with a clean UI sans (Inter, Geist or similar) and **tabular figures** for all currency so numbers align.
- **Components:** 8px spacing grid, about 12px card radius, 1px hairline borders instead of heavy shadows, generous whitespace, a compact density option for tables.
- **Interactions:** quick-add field with live parse preview (chips for amount, category and date appear as you type); optimistic updates; count-up only on the verdict figure; chart draw-in once on load; staggered card entrance under 300 ms; respect `prefers-reduced-motion`.
- **States:** designed empty states with one action (and a "load demo data" option), skeleton loaders shaped like the real content, error states that preserve input and offer retry, a distinct "AI offline" banner.

- **Signature:** the budget-pace hero chart plus the evidence panel that shows *why* any AI statement is true. A visible "Verified by calculation" affordance turns your zero-hallucination architecture into a visual feature.
- **Accessibility:** keyboard-navigable everything, visible focus rings, labelled fields, charts with a data-table toggle, semantic landmarks, 44px touch targets.

G.5 Adapting design guidance to an AI coding environment

Give the coding tool (a) a written **design brief** (this section), (b) **design tokens** in a single file it must import rather than invent values, (c) a short **anti-pattern list** (no default blue/purple gradients, one chart type per concept, no emoji icons, no lorem ipsum), and (d) a **review checklist** you run after each screen: contrast, spacing grid, empty/loading/error states, mobile at 375 px. Generate one screen, review it, then replicate the pattern; do not ask for the whole UI in one prompt.

H. Security and reliability

- **Privacy:** minimize data sent to AI (aggregates, not full ledgers, where possible; no email or name in prompts). Tell users when a hosted model is used. Offer a local-only mode as a stated future option.
- **Auth:** strong hash (argon2 or bcrypt with sensible cost), httpOnly + Secure + SameSite cookies, login rate limiting, generic error messages, minimum password length.
- **Authorization:** `user_id` scoping on every query, tested with cross-user access attempts that must return 404.
- **Input validation:** Pydantic on every request body, bounds on amounts and dates, length limits on text, parameterized queries only (the ORM), CSV export protected against formula injection (prefix cells beginning with `=`, `+`, `-`, `@`).
- **API security:** CORS restricted to your frontend origin, rate limits on AI endpoints (this also protects free quotas), request size limits, no stack traces in responses.
- **Secrets:** environment variables only, `.env` in `.gitignore`, a committed `.env.example`, and **never put AI keys in the frontend**. Check git history before making the repo public.
- **AI risks:** see F.4. Add a persistent disclaimer that insights are informational, not financial advice. Refuse investment, tax and legal queries.

- **Reliability:** DB transactions for bulk saves, idempotency on confirm (no double-save on double-click), timeouts on AI calls, a health endpoint, structured logs, and a seed/reset script so you can restore demo state in seconds.

I. Testing strategy

Priority order (what cannot fail in the demo):

1. Register/login, then seeded data visible.
2. NL entry, review, confirm, dashboard updates.
3. Budget set, utilization and pace correct.
4. Dashboard numbers match known fixtures.
5. Copilot answers a scripted set of questions with evidence; the AI-off path works.

Unit tests (highest value, fastest): the entire `domain/` layer: totals, month boundaries, timezone edge cases (a transaction at 23:30 IST), pace projection, anomaly detection with synthetic spikes and sparse history, recurring detection, paise rounding, and the NL rule parser on 30+ phrases (Indian number formats, "1.5k", "Rs.", relative dates).

Integration tests (API + test DB): auth flow, CRUD, **cross-user isolation** (user B cannot read or modify user A's rows), bulk-save atomicity (one invalid row rolls back all), budget uniqueness, CSV export content, and the AI client against a mocked provider that returns garbage, times out, or returns mismatched numbers (each must trigger fallback).

End-to-end (Playwright or Cypress, 3–5 flows only): the five flows above, run against seeded data. Also run the demo script manually three times, including once with the network disabled.

AI evaluation (small but valuable): a table of about 30 labelled NL-entry examples with expected output; report parse accuracy for rules-only versus rules+LLM. This gives you a concrete number for the presentation.

Manual checks: mobile layout at 375 px, keyboard-only navigation, contrast audit, reduced motion.

J. Development roadmap

Your materials say both four days and three days. **Plan for three working days plus a half-day buffer; if a fourth day is confirmed, spend it on polish, tests and rehearsal, not new features.** *Confirm the deadline with NIVUNA before starting.*

Day 0 (2–3 hours, before coding): finalize the decisions list (Section L.9), set up the repo, design tokens, design-rules file and AI tool instructions, plan seed data, outline the demo script, choose AI providers and test one free-tier call.

Day 1: Foundation and core data

- Morning: backend skeleton, DB models and migrations, auth, seeded categories, transactions CRUD with validation, the `domain/` summary functions with unit tests.
- Afternoon: frontend shell, design system (tokens, typography, buttons, inputs, cards), auth screens, transactions list and manual add.
- Exit criterion: log in, add/edit/delete transactions, see correct totals. Commit and tag.

Day 2: Intelligence and the dashboard

- Morning: budgets, pace logic, dashboard endpoints, hero chart and category bars; seed realistic demo data (3–4 months).
- Afternoon: NL entry (rule parser first, then the LLM stage with fallback), review-and-confirm UI, bulk endpoint.
- Exit criterion: flows W1, W2 and W3 work end to end. Commit and tag.

Day 3: Differentiators and hardening

- Morning: insight engine (pace + anomalies), Copilot with whitelisted tools and evidence panel, AI fallback chain, validator.
- Afternoon: monthly report (template first), empty/loading/error states, mobile pass, accessibility pass, tests for demo-critical flows.
- Exit criterion: the full demo script runs with AI on and AI off.

Half-day buffer or Day 4: deploy and smoke-test, README with setup and architecture diagram, record a backup demo video, write the presentation, rehearse twice with a timer. **Freeze features 6 hours before the presentation.**

Cut order if behind schedule: what-if simulator, then recurring detection, then CSV, then AI narration of the report (keep the template), then Copilot breadth (keep five question types). Never cut: domain-layer tests, seed data, fallbacks, rehearsal.

K. Presentation strategy (about 15–16 minutes plus 13–14 discussion)

K.1 Suggested structure

Minutes	Content
0–1.5	Problem: why trackers get abandoned; who PennyPilot is for (state your assumptions)
1.5–3	Approach and the central design principle: compute, then explain
3–4.5	Architecture diagram and stack rationale (one slide)
4.5–11	**Live demo** (scripted)
11–13	AI-assisted workflow: tools, prompts, what AI generated versus what you changed
13–14.5	Challenges, bugs, what you learned
14.5–16	Future enhancements

Demo script (6–7 minutes): open the seeded dashboard, read the verdict, type a messy NL entry, show the parse preview, edit one low-confidence row, confirm, watch the dashboard and budget update, ask the Copilot a question and open the evidence panel, show an anomaly insight with its evidence, then (optional) toggle AI off and ask again to show the fallback. Keep a recorded backup video.

K.2 Talking points per required topic

- **Why this problem:** relatable, clear data model, and a place where *responsible* AI use is a real design question rather than decoration. Be honest; do not invent a personal story.
- **Stack:** answer each choice as "X because Y; alternative Z rejected because W" (Section D).
- **AI coding tools:** name only tools you actually used and give your real reasons. I cannot know which you used. Keep a **prompt log** during development (date, prompt, tool, result, your edit) so you can answer truthfully.

- **How you wrote prompts:** show one good prompt (context, constraints, acceptance criteria, files to touch, what not to do) and one failed prompt with how you fixed it.
- **What AI generated versus what you reviewed:** be specific, for example scaffolding and boilerplate versus the money-handling logic, timezone edge cases and security checks you wrote or rewrote. Prepare two or three concrete bugs found in AI output (for example float money, unscoped queries, wrong month boundary). Use only real ones; if none occurred, say what you checked.
- **Challenges and learning:** free-tier limits, LLM output reliability, month/timezone boundaries, keeping the UI distinctive within the time limit.
- **Future enhancements:** CSV import with review, bank/UPI statement parsing, local-only mode, multi-currency, notifications, goal planning, a feedback loop that improves categorization.

K.3 Likely discussion questions (with the shape of a good answer)

1. *How do you prevent the LLM from giving wrong numbers?* Compute-then-explain, fact bundles, numeric validator, template fallback.
2. *Why not let the LLM query the database directly?* Injection, hallucinated aggregates and isolation risk; use whitelisted tools instead.
3. *Why integer paise?* Float rounding errors; sums must be exact.
4. *How do you isolate users' data?* Required `user_id` in repositories, UUIDs, cross-user tests.
5. *What happens if the AI provider is down or rate-limited?* Fallback chain, timeouts, visible state, rules-only parser.
6. *Why FastAPI, SQL and React rather than X?* Section D reasoning.
7. *How does anomaly detection work, and what are its limits?* Median/MAD, minimum history, false positives, it cannot explain causes.
8. *What would break at 10,000 users?* Aggregation queries (indexes, summary tables), AI quota (queueing and caching), auth hardening; none are needed now.
9. *How did you test the AI features?* Labelled set, mocked providers, failure-injection tests.
10. *What is the privacy impact of sending data to a hosted model?* Minimization, user disclosure, local mode.
11. *How do you handle timezones and month boundaries?* Dates stored in the user's timezone, tests at 23:30 IST.
12. *What did the AI get wrong while building this?* Prepare real examples.

13. *What did you decide not to build, and why?* The cut list, reasoned by value versus risk.
14. *How is this different from existing apps?* The Section A table.

L. Final summary

L.1 Recommended final MVP

Auth; transactions CRUD with filters; categories; monthly category budgets with pace; dashboard with verdict strip, pace hero chart and category bars; **natural-language entry with review-before-save**; three insight types (budget pace, anomaly, category shift); grounded Copilot with five question types and an evidence panel; template monthly report; seeded demo data; full AI-off fallback; CSV export if time allows.

L.2 Five features that make it stand out

1. NL entry with live parse preview and mandatory confirmation.
2. Budget **pace** projection rather than only utilization.
3. Grounded Copilot with an **evidence panel** (visible proof of no fabrication).
4. Explainable anomaly insights that show baselines and suppress themselves when data is thin.
5. Graceful degradation: the whole app works with AI off, and says so.

L.3 Technology stack

React + TypeScript + Vite + Tailwind (token-driven) + TanStack Query + Recharts (plus a custom SVG hero chart); FastAPI + Pydantic + SQLAlchemy + Alembic; SQLite locally and managed free Postgres for the hosted copy; cookie-based JWT or sessions with argon2/bcrypt; provider-abstracted LLM client (free hosted first, optional Ollama, deterministic fallback); Pytest, Vitest and Playwright.

L.4 Recommended architecture

Single-page frontend, one modular FastAPI service, relational DB. `domain/` pure financial logic is the single source of numeric truth. `ai/` consumes only domain outputs and is wrapped in timeout, schema validation, numeric validation and fallback. No queues, caches, vector stores or microservices.

L.5 Zero-budget AI implementation strategy

Rules and statistics for everything computable; an LLM only for language understanding (NL parsing residue, Copilot intent, narrative phrasing). Chain: free hosted provider, then a second provider or local model, then templates. Rate-limit AI endpoints to protect free quotas, cache insight narratives per (user, month, facts hash), and keep a recorded demo as insurance. All provider limits and data terms must be re-checked before the build.

L.6 Visual design direction

"Quiet Ledger": warm neutral surfaces, a single deep green-teal accent, editorial display type with tabular-figure UI sans, hairline borders, a verdict-first dashboard with one hero chart, evidence panels, designed empty/loading/error/AI-offline states, reduced-motion-aware animation, AA contrast.

L.7 Realistic schedule

Day 0 setup and decisions (2–3 h); Day 1 data foundation and core CRUD; Day 2 budgets, dashboard, NL entry; Day 3 insights, Copilot, hardening; half-day buffer (or Day 4) for deploy, README, backup video and rehearsal. Feature freeze 6 hours before presenting.

L.8 Biggest technical risks

1. **Live-demo failure** from free-tier cold starts or network (mitigate: local primary, warm backup, recorded video).
2. **LLM unreliability and quota exhaustion** (mitigate: fallbacks, validator, rate limits).
3. **Scope creep** across nine candidate features (mitigate: the cut order in Section J).
4. **Date, timezone and month-boundary bugs** in aggregates (mitigate: domain tests).
5. **AI-generated code you cannot explain** (mitigate: review every money, auth and query-scoping function; keep the prompt log).
6. **Design polish consuming the schedule** (mitigate: tokens first, one screen as a template, budgeted polish time).
7. **Anomaly false positives** on sparse data (mitigate: minimum-history rule, realistic seed data).

L.9 Decisions to finalize before implementation

1. Confirmed submission deadline and presentation date/time (three versus four days).

2. Hosted versus local-first demo plan, and which free hosts, after verifying current terms.
3. Primary and fallback AI providers, after verifying free-tier limits and data-use terms; whether Ollama is feasible on your hardware.
4. Database for the hosted copy (SQLite with persistent disk versus managed Postgres).
5. Auth method (cookie JWT versus server sessions) and whether to ship a seeded demo user alongside registration.
6. Overall monthly budget model (separate field versus null-category row).
7. Whether to include goals and the what-if simulator, or cut them now.
8. Final differentiator set (my recommendation: NL entry, pace budgets, Copilot with evidence, anomalies, graceful degradation).
9. Which design rules from the three reference repos you adopt (requires actually reading them), and the final palette and fonts after checking font licences.
10. Which AI coding tools you actually use, and the prompt-log format.
11. Seed data design: realistic Indian spending patterns across 3–4 months, including deliberate anomalies for the demo.
12. Copilot's supported question list (five to eight types) and its refusal policy.
13. Chart library final choice, and whether the hero chart is custom SVG.
14. Test scope you commit to (domain layer fully covered, five E2E flows, a 30-example NL evaluation set).

Items marked "to verify" were not checked against live sources in this document. Review this blueprint critically; it reflects judgement, not guarantees.